

1 Data Preprocessing and Detection Models Implementation

1.1 Introduction

In the context of the **AEGIS Entropy** project, the artificial intelligence-based detection module constitutes the core component of the defense system against *Port Scan* attacks. This phase of the project aims to design, implement, and evaluate a comprehensive Machine Learning pipeline, strictly adhering to the PDCA (*Plan-Do-Check-Act*) methodology and the academic standards taught in class.

The primary objective is to build a detection pipeline capable of achieving the success criteria defined in the project's final plan:

- **F1-Score** $\geq 88\%$
- **Accuracy** $\geq 90\%$
- **False Positive Rate (FPR)** $< 6\%$

This chapter details every step of the pipeline: from exploring and cleaning the raw data, through selecting features according to the Data Dictionary, to the training, cross-validation, and final evaluation of the three selected models: **Random Forest**, **XG-Boost**, and **Isolation Forest**.

1.2 Dataset Used

The dataset used for training and evaluating the models is **CICIDS2017** (*Canadian Institute for Cybersecurity Intrusion Detection System*), an academic benchmark dataset for network intrusion detection. It was generated in a controlled environment replicating realistic network traffic conditions, including both benign traffic and simulated attacks.

The specific file utilized is:

`Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv`

This file contains network flows captured on Friday afternoon, a period during which *Port Scan* attacks (via **Nmap**) were executed.

Table 1: Dataset characteristics

Property	Value
Number of instances (rows)	286,467
Number of features (columns)	79
PortScan class	158,930 (55.48%)
BENIGN class	127,537 (44.52%)
Format	CSV (aggregated network flows)

A slight imbalance in favor of the **PortScan** class is observed, which will be addressed later using the SMOTE technique (Section 1.5.3).

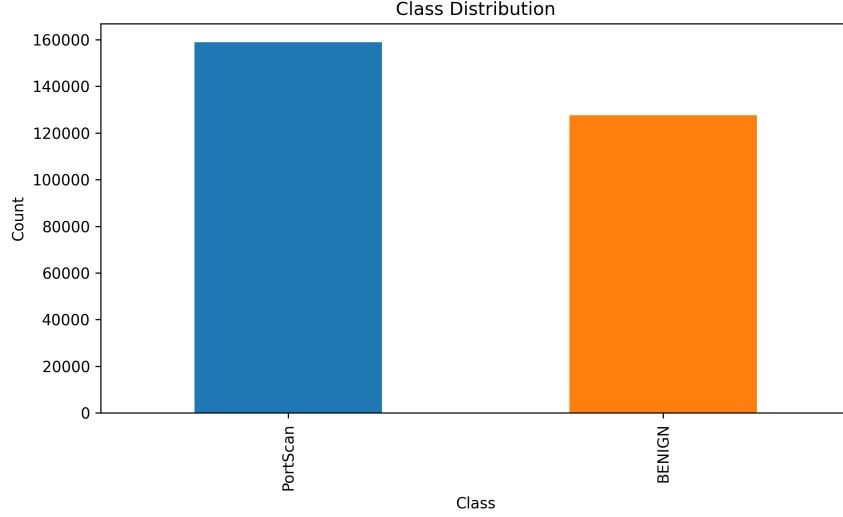


Figure 1: Class distribution in the CICIDS2017 dataset (PortScan vs BENIGN)

1.3 Feature Selection (Data Dictionary)

The project’s Data Dictionary defines **13 features** deemed relevant for detecting Port Scan attacks. However, since the CICIDS2017 dataset is a collection of aggregated flow data (*flow-level*), it does not directly contain all these features. A **mapping** process was conducted to establish the correspondence between the theoretical features and the available CSV columns.

Table 2: Feature Mapping: Data Dictionary → CSV Columns

Feature (Dict.)	Avail.	CSV Column	Justification
Distinct Dst Ports	≈	Destination Port	Proxy: the variance of ports targeted by scanners provides a discriminative signal
SYN Flag Count	✓	SYN Flag Count	Direct match
RST Flag Count	✓	RST Flag Count	Direct match
Flow Duration	✓	Flow Duration	Direct match
Total Fwd Packets	✓	Total Fwd Packets	Direct match
ACK Flag Count	✓	ACK Flag Count	Direct match
IAT Mean	✓	Flow IAT Mean	Direct match
Bwd Packet Length	✓	Bwd Packet Length Mean	Mean variant
TCP Window Size	✓	Init_Win_bytes_forwarded	Best proxy available in CICIDS2017
Unique Dst IPs	×	—	Absent from CSV; live capture feature only
TTL Value	×	—	Absent from flow-level data

Feature (Dict.)	Avail.	CSV Column	Justification
Shadow Node Interaction	N/A	Placeholder = 0	Future integration with honeypot module
MTD Port Delta	N/A	Placeholder = 0	Future integration with MTD module

Final Decision: The pipeline uses **11 features** in total: 9 extracted from the CSV + 2 placeholders (initialized to 0) reserved for real-time integration with the honeypot and MTD modules.

```

1 features = [
2     'Destination Port',          # Proxy for Distinct Dst Ports
3     'Flow Duration',
4     'Total Fwd Packets',
5     'SYN Flag Count',
6     'RST Flag Count',
7     'ACK Flag Count',
8     'Flow IAT Mean',
9     'Bwd Packet Length Mean',
10    'Init_Win_bytes_forward'     # Proxy for TCP Window Size
11 ]
12 X = df[features].copy()
13 X['shadow_node_interaction'] = 0 # Honeypot placeholder
14 X['mtd_port_delta'] = 0         # MTD placeholder

```

Listing 1: Defining features in the pipeline

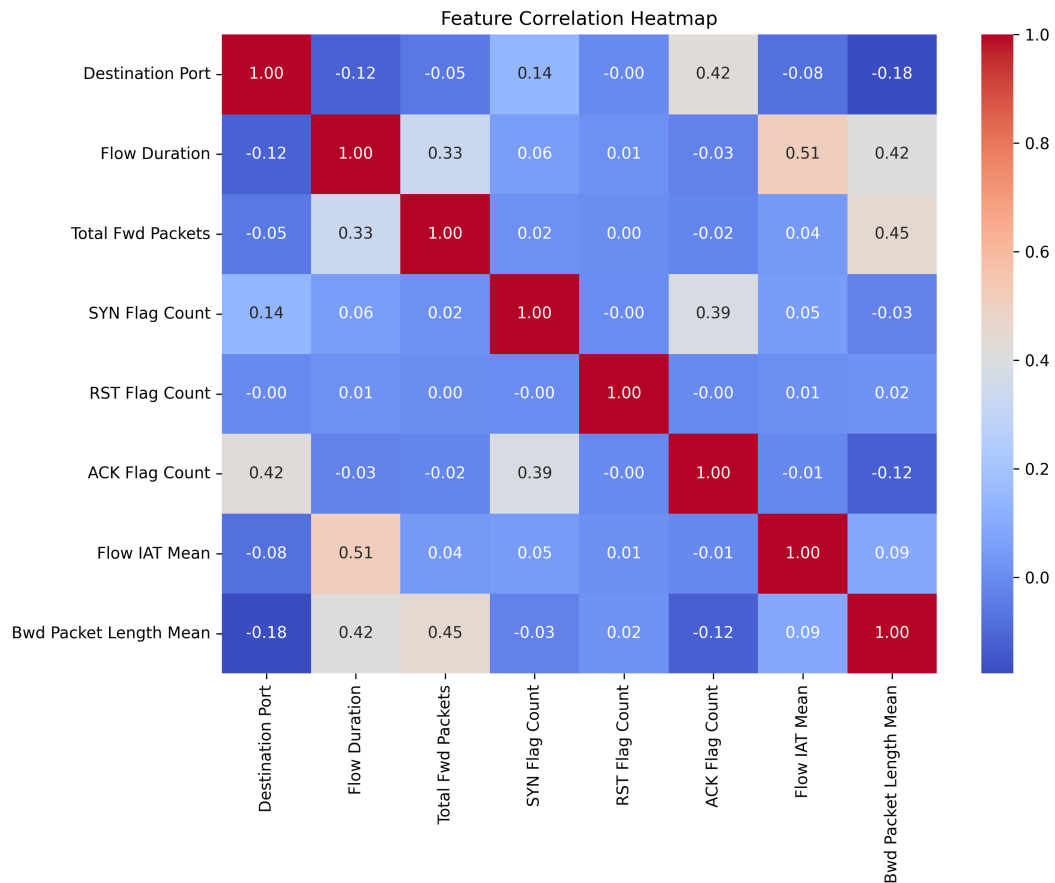


Figure 2: Correlation matrix between selected features

1.4 Preprocessing Pipeline

Data preprocessing is a critical step that directly influences model quality. Following the methodology taught in class, the pipeline follows a sequential four-step process: cleaning, outlier detection, skewness correction, and export.

1.4.1 Data Cleaning

The first step involves preparing the raw data for analysis. The CICIDS2017 dataset presents several common issues:

1. **Spaces in column names:** CSV headers contain leading spaces (e.g., ' Flow Duration' instead of 'Flow Duration'). These spaces cause silent errors when accessing columns.
2. **Infinite values:** Some columns contain `inf` or `-inf` values, resulting from division by zero during network flow calculations.
3. **Missing values:** After replacing infinities, `NaN` values may remain.

```

1 # 1. Remove spaces in column names
2 df.columns = [col.strip() for col in df.columns]
3

```

```

4 # 2. Replace infinite values with NaN
5 df.replace([np.inf, -np.inf], np.nan, inplace=True)
6
7 # 3. Imputation by median (robust to outliers)
8 df[FEATURES] = df[FEATURES].fillna(df[FEATURES].median())

```

Listing 2: Cleaning raw data

Choice of median: Unlike the mean (sensitive to extreme values) or zero replacement (which introduces bias), the median provides a robust estimation of central tendency, particularly suited for the asymmetric distributions typical of network traffic.

1.4.2 Outlier Detection and Handling (IQR Method)

Outliers can distort model training by creating biases in distributions. The **Interquartile Range (IQR)** method is used to detect them in a statistically rigorous manner.

Formal Definition: Let Q_1 be the first quartile (25th percentile) and Q_3 be the third quartile (75th percentile) of a variable. The interquartile range is defined by:

$$IQR = Q_3 - Q_1 \quad (1)$$

The detection bounds are:

$$\text{Lower Bound} = Q_1 - 1.5 \times IQR \quad ; \quad \text{Upper Bound} = Q_3 + 1.5 \times IQR \quad (2)$$

Any observation outside the interval [Lower Bound, Upper Bound] is considered an outlier.

Handling Strategy: Rather than **removing** rows containing outliers (which risks losing valuable attack data), we apply **capping** (*Winsorization*): values beyond the bounds are adjusted to the nearest bound.

```

1 for feature in FEATURES:
2     Q1 = df[feature].quantile(0.25)
3     Q3 = df[feature].quantile(0.75)
4     IQR = Q3 - Q1
5     lower_bound = Q1 - 1.5 * IQR
6     upper_bound = Q3 + 1.5 * IQR
7
8     # Count detected outliers
9     outliers = ((df[feature] < lower_bound) |
10                (df[feature] > upper_bound)).sum()
11
12     # Capping instead of removal
13     df[feature] = np.where(
14         df[feature] < lower_bound, lower_bound, df[feature])
15     df[feature] = np.where(
16         df[feature] > upper_bound, upper_bound, df[feature])

```

Listing 3: Outlier detection and capping using IQR

Table 3: Number of outliers detected per feature (IQR method)

Feature	Detected Outliers
Destination Port	37,788
Flow Duration	52,212
Total Fwd Packets	35,156
SYN Flag Count	6,014
RST Flag Count	21
ACK Flag Count	35,555
Flow IAT Mean	57,303
Bwd Packet Length Mean	32,387
Init_Win_bytes_forward	0

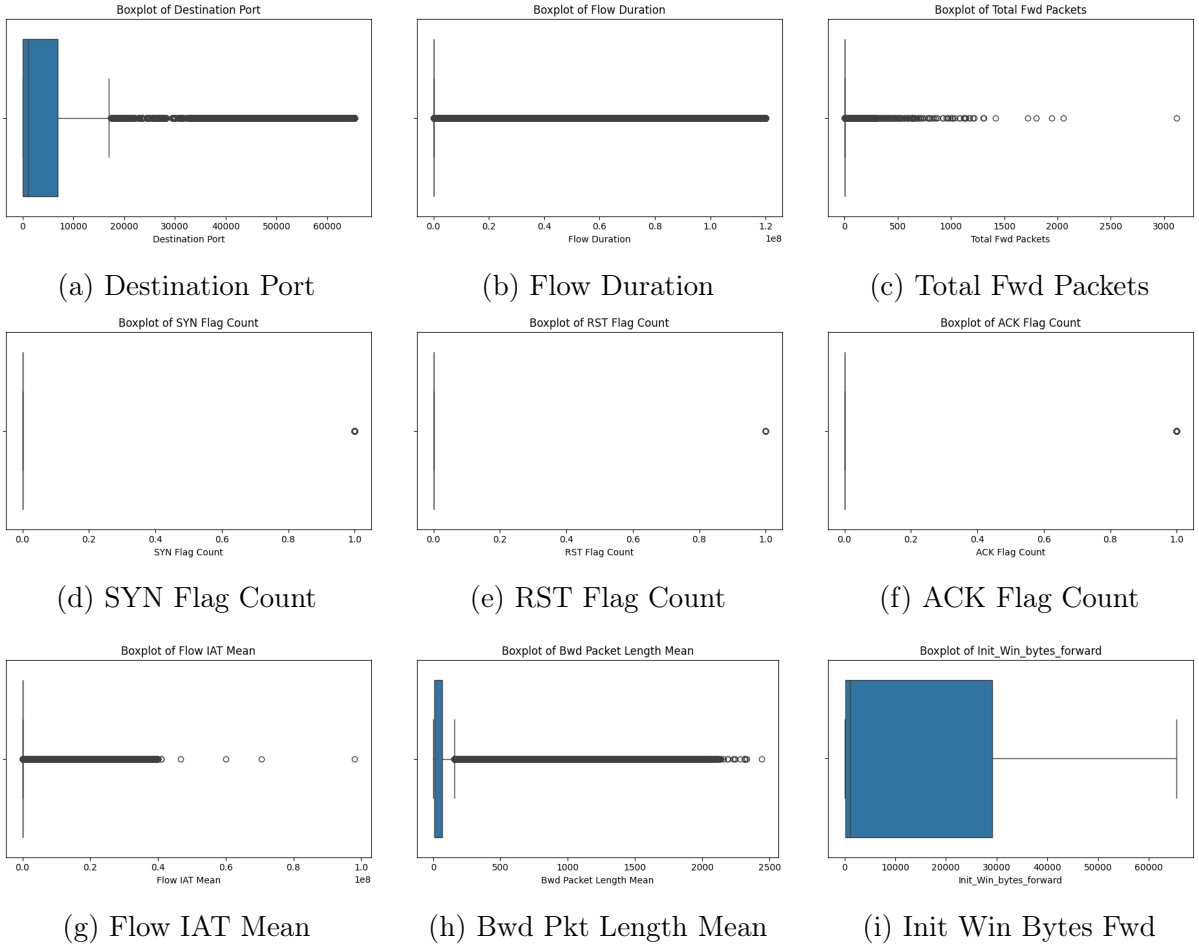


Figure 3: Boxplots of all 9 selected features illustrating the presence and extent of outliers

1.4.3 Skewness Analysis and Logarithmic Transformation

Skewness measures the degree of asymmetry of a distribution relative to a normal distribution. Fisher's coefficient is defined by:

$$\gamma_1 = \frac{E[(X - \mu)^3]}{\sigma^3} \quad (3)$$

where μ is the mean and σ the standard deviation. A value of $|\gamma_1| > 1$ indicates strong skewness, justifying a transformation to normalize the distribution.

Applied Transformation: For features exhibiting strong skewness ($|\text{skew}| > 1.0$), we apply the logarithmic transformation $\log(1 + x)$ (function `np.log1p`). Using $\log(1 + x)$ rather than $\log(x)$ is essential because it:

- Properly handles zero values ($\log(1 + 0) = 0$)
- Preserves the relative order of observations
- Reduces the influence of extreme values on the distribution

```

1 for feature in FEATURES:
2     skewness = df[feature].skew()
3     if abs(skewness) > 1.0:
4         # Shift if negative values exist
5         min_val = df[feature].min()
6         if min_val < 0:
7             df[feature] = df[feature] - min_val
8         # Log(1+x) transformation
9         df[feature] = np.log1p(df[feature])

```

Listing 4: Skewness analysis and logarithmic transformation

Table 4: Feature skewness analysis

Feature	Skewness	Action
Destination Port	1.29	Log transform applied
Flow Duration	1.27	Log transform applied
Total Fwd Packets	1.32	Log transform applied
SYN Flag Count	0.00	None
RST Flag Count	0.00	None
ACK Flag Count	0.00	None
Flow IAT Mean	1.24	Log transform applied
Bwd Packet Length Mean	1.27	Log transform applied
Init_Win_bytes_forward	0.96	None (< 1.0)

1.4.4 Preprocessed Dataset Export

After all transformations, the cleaned dataset is exported in CSV format to ensure the **reproducibility** of the pipeline. The training and evaluation scripts directly consume this preprocessed file.

```

1 df.to_csv("../data/preprocessed.csv", index=False)

```

Listing 5: Preprocessed dataset export

1.5 Data Splitting and Normalization

1.5.1 Train/Test Split (80/20)

The preprocessed dataset is divided into a training set (80%) and a test set (20%), using **stratified sampling** to preserve class proportions in each partition.

```
1 X_train, X_test, y_train, y_test = train_test_split(  
2     X, y,  
3     test_size=0.2,  
4     random_state=42,      # Reproducibility  
5     stratify=y            # Preserves class proportions  
6 )  
7 # Result: Train = 229,173 | Test = 57,294
```

Listing 6: Stratified dataset splitting

1.5.2 Standardization (StandardScaler)

Standardization transforms each feature to have a mean of 0 and a standard deviation of 1, according to the formula:

$$z = \frac{x - \mu}{\sigma} \quad (4)$$

where μ and σ are respectively the mean and standard deviation of the feature **calculated solely on the training set**.

Data Leakage Prevention: It is imperative that the `StandardScaler` is fitted (`fit`) **exclusively on the training data**. If the scaler were fitted on the entire dataset (train + test), the test set statistics would contaminate the learning process, skewing evaluation results. The scaler is then saved to be reused during production inference.

```
1 scaler = StandardScaler()  
2 # fit() on train ONLY, then transform() on both  
3 X_train_scaled = scaler.fit_transform(X_train)  
4 X_test_scaled  = scaler.transform(X_test)  
5  
6 # Save for production inference  
7 joblib.dump(scaler, "../models/scaler.pkl")
```

Listing 7: Standardization with data leakage prevention

1.5.3 Class Balancing (SMOTE)

Although the imbalance is moderate (55/45), applying **SMOTE** (*Synthetic Minority Oversampling Technique*) ensures unbiased training. SMOTE generates synthetic examples of the minority class by interpolating between existing nearby instances in the feature space.

SMOTE Principle: For each instance of the minority class, SMOTE:

1. Identifies its k nearest neighbors (default $k = 5$)

2. Randomly selects one of these neighbors
3. Creates a synthetic point on the line segment connecting the original instance to the selected neighbor

Application on training data only: SMOTE is applied **after** splitting and standardization, and **only** on the training set. Applying SMOTE before the split would introduce data leakage, as synthetic points generated from test data could end up in the training set.

```
1 from imblearn.over_sampling import SMOTE
2
3 smote = SMOTE(random_state=42)
4 X_train_res, y_train_res = smote.fit_resample(
5     X_train_scaled, y_train
6 )
7 # Before SMOTE: 229,173 samples (imbalanced)
8 # After SMOTE: 254,288 samples (balanced)
```

Listing 8: Application of SMOTE on the training set

1.6 Model Implementation

Three models were selected in accordance with the project's final plan, each providing a complementary approach to detection:

1.6.1 Random Forest

Random Forest is a supervised learning algorithm based on the *bagging* (*Bootstrap Aggregating*) principle. It builds an ensemble of n decision trees, each trained on a random sub-sample of the data with a random subset of features. The final prediction is obtained by **majority vote**.

Advantages for Port Scan detection:

- Robust to overfitting due to tree diversity
- Effectively handles high-dimensional data
- Provides a measure of *feature importance*

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rf = RandomForestClassifier(
4     n_estimators=100,      # 100 decision trees
5     random_state=42
6 )
7 rf.fit(X_train_res, y_train_res)
8 joblib.dump(rf, "../models/random_forest.pkl")
```

Listing 9: Random Forest Training

1.6.2 XGBoost (eXtreme Gradient Boosting)

XGBoost is a *gradient boosting* algorithm that builds decision trees **sequentially**, with each new tree correcting the errors of the previous ones. It integrates regularization (L_1 and L_2) to limit overfitting.

Advantages for Port Scan detection:

- Considered state-of-the-art for tabular data
- Gradient descent optimization with regularization
- Native handling of missing values

```
1 from xgboost import XGBClassifier
2
3 xgb = XGBClassifier(
4     n_estimators=100,
5     learning_rate=0.1,      # Learning rate
6     max_depth=6,           # Maximum tree depth
7     random_state=42,
8     eval_metric="logloss"
9 )
10 xgb.fit(X_train_res, y_train_res)
11 joblib.dump(xgb, "../models/xgboost.pkl")
```

Listing 10: XGBoost Training

1.6.3 Isolation Forest

Isolation Forest is an **unsupervised** anomaly detection algorithm. Its principle is based on the idea that anomalies, being "few and different", are easier to isolate through random partitioning. The quicker a point is isolated (lower depth in the tree), the more likely it is to be an anomaly.

Role in AEGIS Entropy: This algorithm was selected for its theoretical ability to detect "**Low and Slow**" (slow and stealthy) attacks, which evade supervised models trained on aggressive attack signatures.

```
1 from sklearn.ensemble import IsolationForest
2
3 iso = IsolationForest(
4     contamination='auto', # Automatic threshold
5     random_state=42
6 )
7 iso.fit(X_train_scaled) # Without SMOTE (unsupervised)
8 joblib.dump(iso, "../models/isolation_forest.pkl")
```

Listing 11: Isolation Forest Training

1.7 Stratified K-Fold Cross-Validation

To ensure the statistical reliability of the results and avoid any dependence on a specific data split, a **10-fold Stratified Cross-Validation** (*Stratified K-Fold*, $k = 10$) was implemented.

Principle: The training dataset is divided into 10 equal-sized subsets (*folds*). In each iteration, 9 folds are used for training and the 10th is used for validation. The process is repeated 10 times, and the final score is the average of the 10 scores. The **stratified** option ensures that each fold preserves the original class proportions.

```

1 from sklearn.model_selection import StratifiedKFold,
   cross_val_score
2
3 cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42)
4
5 rf_scores = cross_val_score(
6     rf, X_train_res, y_train_res,
7     cv=cv, scoring='f1', n_jobs=-1
8 )
9
10 xgb_scores = cross_val_score(
11     xgb, X_train_res, y_train_res,
12     cv=cv, scoring='f1', n_jobs=-1
13 )

```

Listing 12: 10-Fold Stratified Cross-Validation

Table 5: Stratified cross-validation results (F1-Score)

Model	Mean F1	Standard Deviation ($\pm 2\sigma$)
Random Forest	0.9993	± 0.0003
XGBoost	0.9994	± 0.0003

Note: Isolation Forest, being an unsupervised algorithm, cannot be evaluated via traditional cross-validation (it does not directly output an F1 score).

1.8 Evaluation and Results

1.8.1 Metrics Used

Model evaluation relies on six complementary metrics, each capturing a different aspect of performance:

- **Accuracy:** overall proportion of correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

- **Precision:** out of instances predicted as attacks, how many actually are.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (6)$$

- **Recall**: out of actual attacks, how many were detected.

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7)$$

- **F1-Score**: harmonic mean of precision and recall.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (8)$$

- **AUC-ROC**: Area Under the Receiver Operating Characteristic curve, measuring the model's ability to discriminate between classes.
- **FPR** (False Positive Rate): proportion of benign traffic incorrectly classified as an attack.

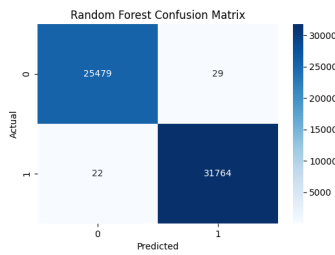
$$\text{FPR} = \frac{FP}{FP + TN} \quad (9)$$

where TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.

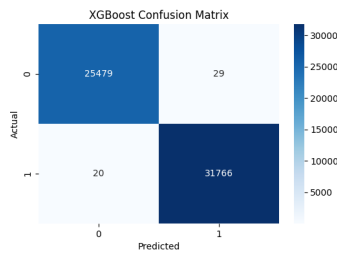
1.8.2 Comparative Results

Table 6: Performance comparison of the three models

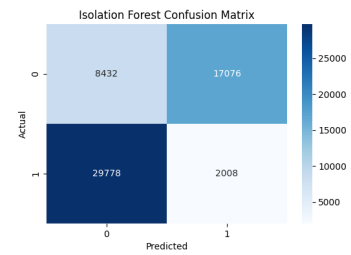
Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	FPR
Random Forest	99.91%	99.91%	99.93%	99.92%	0.9997	0.11%
XGBoost	99.91%	99.91%	99.94%	99.92%	0.9999	0.11%
Isolation Forest	18.22%	10.52%	6.32%	7.89%	N/A	66.94%



(a) Random Forest



(b) XGBoost



(c) Isolation Forest

Figure 4: Confusion matrices of the three models on the test set

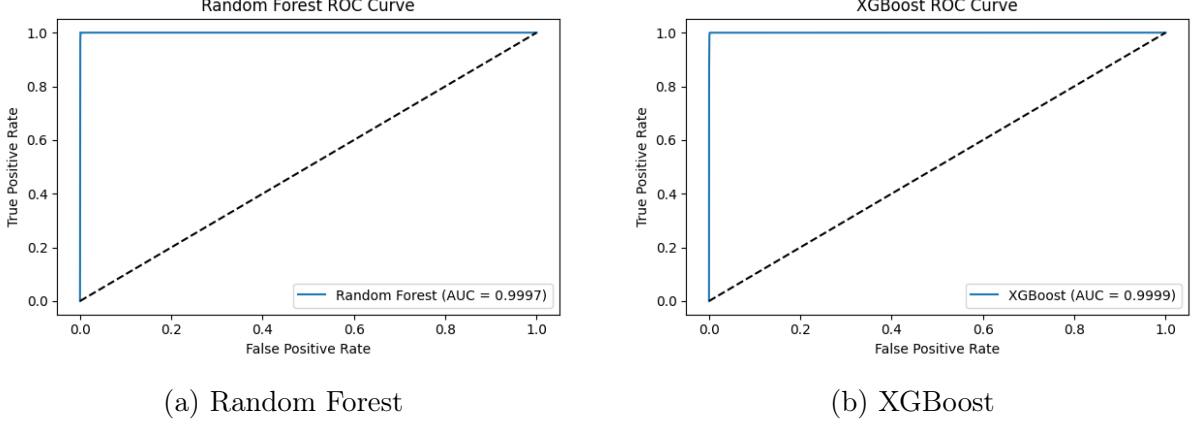


Figure 5: ROC curves of the supervised models (Random Forest and XGBoost)

1.8.3 Success Criteria Verification

The project’s final plan defines three performance thresholds that the detection system must achieve. The table below compares the obtained results against these criteria:

Table 7: Verification of project success criteria

Criterion	Threshold	RF	Status	XGBoost	Status
F1-Score	$\geq 88\%$	99.92%	PASS	99.92%	PASS
Accuracy	$\geq 90\%$	99.91%	PASS	99.91%	PASS
FPR	$< 6\%$	0.11%	PASS	0.11%	PASS

Both supervised models (Random Forest and XGBoost) significantly exceed all defined success criteria.

1.8.4 Analysis of Isolation Forest’s Failure

Isolation Forest was integrated into the architecture for its theoretical capacity to detect stealthy "**Low and Slow**" attacks. However, empirical results on the CICIDS2017 dataset reveal very poor performance (F1-Score of 7.89%, FPR of 66.94%). This under-performance is explained by two fundamental factors:

1. The Majority Anomaly Paradox: Isolation Forest is mathematically designed to isolate data points that are "*few and different*", i.e., anomalies typically representing less than 5% of the dataset. However, in our CICIDS2017 dataset, the **PortScan** class constitutes **55.48%** of the traffic — it is the *majority* class. The algorithm, trained on the complete set (normal traffic + attacks), is unable to distinguish attacks as anomalies since they represent the statistical norm of the dataset.

2. Mismatch between Aggressive and Stealthy Signatures: The port scans present in the CICIDS2017 dataset are primarily aggressive **Nmap** scans, generating a massive burst volume of packets. These signatures are dense and do not match the "Low and Slow" profile (spaced flows over time, low volume) for which Isolation Forest was selected.

Conclusion: This failure does not undermine the architectural relevance of Isolation Forest in the AEGIS Entropy system. Instead, it demonstrates a methodological limitation: to be effective in anomaly detection, Isolation Forest must be trained exclusively on verified **BENIGN** traffic (semi-supervised approach) in order to establish a clean baseline of normal network behavior. In a production environment where attacks are genuinely rare and stealthy, this model would regain its relevance. For detecting the aggressive scans present in this dataset, the supervised models (Random Forest and XGBoost) are vastly superior.

1.9 Conclusion

This chapter presented the complete design and implementation of the machine learning detection pipeline for the AEGIS Entropy system. The pipeline, built according to a rigorous methodology, includes:

1. **Comprehensive data preprocessing** (cleaning, IQR outlier detection, skewness correction via logarithmic transformation)
2. **Normalization** via StandardScaler with strict data leakage prevention
3. **Class balancing** via SMOTE applied solely on the training set
4. The **training** of three complementary models (Random Forest, XGBoost, Isolation Forest)
5. A **10-fold Stratified Cross-Validation** confirming the stability of results
6. A comprehensive **evaluation** using six metrics and verification of success criteria

The results demonstrate that the supervised models achieve exceptional performance: an F1-Score of **99.92%**, an Accuracy of **99.91%**, and an FPR of only **0.11%**, vastly exceeding the thresholds required by the project's final plan. The analysis of Isolation Forest identified the necessary conditions for its effective deployment in a real-world environment.

The pipeline is now ready for integration with the real-time capture modules, the honeypot system, and the monitoring dashboard, which will be the subjects of the subsequent chapters.